Building and Deploying a Dataspace: A Blueprint for structuring a Dataspace Project

1st Michael Spiegel, BSc
Computer Science
Vorarlberg University of Applied Sciences
Dornbirn, Austria
michael.spiegel@students.fhv.at

***Abstract*—The concept of dataspaces was introduced in the year 2005 by [1]. Since then, the accelerating digital transformation has reshaped the role of data within enterprises. This is creating new opportunities for data-driven innovation and sharing within ecosystems. Dataspaces address challenges such as trust, interoperability, and data sovereignty, offering data exchange between participants. This paper focuses on simplifying the process of creating new dataspace projects. It also focuses on the deployment of dataspaces, demonstrating how this process can be implemented using well-established tools and technologies. A proof of concept was developed to simplify the creation of new dataspace projects. The paper provides practical examples and templates for simple and flexible deployment based on Continuous Integration/Continuous Delivery pipelines using tools such as Docker and Kubernetes. The proof of concept of the project initializer serves as a starting point for creating dataspace projects. Docker Compose and Kubernetes are feasible solutions for deploying dataspaces without encountering significant issues.**

***Index Terms*—Dataspaces, Deployment, Eclipse Dataspace Components, Gaia-X, International Dataspaces Association**

I. Introduction

The accelerating digital transformation is reshaping the role of data in enterprises, creating new opportunities for data-driven innovation through data sharing within ecosystems. Aligning individual and ecosystem interests requires addressing challenges such as trust, interoperability, and data sovereignty, needs that dataspaces are designed to meet. Initiatives like Gaia-X and International Data Spaces Association (IDSA) offer reference architectures for building these distributed data infrastructures. While their potential is recognized by businesses and policymakers, further research is needed to understand how dataspaces can be effectively designed and developed [2, p. 3].

The objective of this paper is to design and implement an easily deployable dataspace that not only functions effectively but also serves as a blueprint for future implementations. The focus is on developing a standard-compliant and reproducible setup that facilitates broader adoption.

This research is guided by the following key question:

- How can a blueprint for an easily deployable dataspace be developed based on existing standards, such as Gaia-X?

By addressing these issue, this paper contributes to the wider effort to make dataspaces more accessible and functional, enabling organizations to realise the full potential of secure and efficient data sharing.

A. Motivation

The increasing reliance on data-driven Artificial Intelligence (AI) is transforming various industries, including healthcare, manufacturing, and transportation, by improving efficiency and enabling innovative solutions. However, the free flow of data is often hindered by concerns over data privacy and ownership. Data sharing platforms, like industrial dataspaces and personal dataspaces, are emerging as critical solutions to address these challenges by providing secure, trusted environments for data exchange. These initiatives offer a pathway for businesses to unlock the full potential of proprietary and personal data while ensuring privacy and regulatory compliance [3, p. 5].

Recognizing the transformative potential of data sharing, the European Union (EU) emphasizes its role in enabling industries to develop innovative products and services while also providing essential resources for training AI systems. To foster this data-driven innovation, the EU introduced the Data Governance Act and the Data Act. The Data Governance Act aims to promote the secure and trusted exchange of data across sectors, facilitating data access and use while ensuring privacy and compliance with regulations [4]. Meanwhile, the Data Act focuses on ensuring fair and transparent data access, addressing issues related to data ownership, and strengthening data portability to unlock the full value of data in the European economy [5]. Together, these initiatives provide a comprehensive framework to support the growth and integration of dataspaces, driving the digital transformation across industries.

An example of a data-sharing ecosystem that is already in use is the Mobility Data Space (MDS)[1], enabling the secure and sovereign exchange of mobility-related data. Supported by the German Federal Ministry for Digital and Transport and aligned with the Gaia-X initiative, the MDS connects stakeholders such as transport companies, automotive manufacturers, and public authorities. By fostering collaboration in a trusted environment, it facilitates data-driven innovation in traffic management, smart mobility solutions, and sustainable transportation.

[1] https://mobility-dataspace.eu/de/

B. Problem Statement

The Eclipse Dataspace Components (EDC)[2] project consists of various components that are actively being developed to facilitate dataspace management and interoperability. Despite the progress made, the setup and deployment process for these components remains unclear and challenging. The lack of straightforward deployment instructions or frameworks complicates adoption, particularly for those who are new to the technology or seeking to integrate EDC into existing infrastructures.

Complementing these developments, the Minimum Viable Dataspace (MVD)[3] serves as a demonstration platform and experimental environment, designed to showcase EDC's capabilities. While it successfully integrates all the essential components needed for a functional dataspace, its current deployment methods which are primarily through IntelliJ or a single Kubernetes cluster are not practical for real-world applications. The reliance on a single Kubernetes cluster for all components fails to reflect a realistic deployment scenario. Additionally, the MVD is not production-ready, posing further challenges for those aiming to implement EDC solutions in operational environments.

C. Objectives

As stated in the beginning of section I, this paper aims to design and implement an easily deployable dataspace that serves as both a functional solution and a blueprint for future implementations. The research questions outlined earlier guide this objective by focusing on how a standard-compliant and reproducible dataspace can be developed based on existing frameworks like Gaia-X. Ensuring ease of deployment and broader adoption is a key goal, allowing both companies and research institutions to integrate the solution seamlessly into their infrastructures.

To achieve this, the paper will present a proof of concept for a project initializer that streamlines the process of creating dataspace projects. It will present a simple and flexible deployment process that uses Continuous Integration/Continuous Deployment (CI/CD) pipelines to demonstrate how dataspaces can be deployed using well-established tools. Additionally, deployment templates will be provided.

Furthermore, this paper will provide an overview of the current state of dataspaces, including existing implementations such as the MVD and the MDS. By analyzing their deployment strategies and identifying their limitations, the research will propose a practical and real-world deployment framework. The outcome will serve as a reference model for organizations looking to adopt dataspace technologies, contributing to the advancement of secure and scalable data-sharing infrastructures.

II. Background

This sections covers the theoretical and technical background. The theoretical background provides an overview of the state of the art of dataspaces. This includes the concept of dataspaces and the importance of the concept for digital ecosystems, as well as an overview of use cases and real-world applications. It also provides an introduction to access and usage control in the context of dataspaces and presents existing frameworks. There will also be an overview of standards and frameworks in place for dataspaces. The technical background explains in detail how dataspaces work.

A. State of the Art

This section explains the definition of dataspaces and their importance for digital ecosystem. It also explores use cases and existing dataspaces for real-world applications. An introduction to access and usage control, as well as an overview of existing frameworks is given. Finally, existing standards and frameworks for dataspaces are presented.

1) Concept of Dataspaces: The topic dataspaces was first introduced by [1] as a new abstraction for data management. They mention that developers where faced with many loosely connected data sources rather than a single data management system. Their idea was that a dataspace is not a data integration approach but more of a data co-existence approach. Furthermore they use the term DataSpace Support Platform (DSSP) when explaining dataspace systems. According to [1] the distinguishing properties of a DSSP they state are the following:

- The DSSP has to support all data in the dataspace and needs to deal with data and applications in a wide variety of formats.
- Even though a DSSP offers different services for administering the dataspace the data still needs to be accessible and modifiable through an interface that is native to the system that is hosting the data.
- A DSSP may be capable to produce the best results it can also if some of the data sources are not available.
- There should be tools offered to create tighter integration of data in the space as necessary.

The following principles can be used to describe the concept of a dataspace, as there is no generally accepted definition for the term dataspace [6, pp. 8–9]. In general, a dataspace consists of the principles of sharing, governance, data sovereignty, openness and federation/interoperability. [6] explain the principles as follows:

- **Sharing:** Data sharing is a core concept of a dataspace. Data sharing can take a number of forms, for example, data can be shared through a transfer. Another form is access to data in a way that it is limited in time, content and scope.
- **Governance:** A dataspace contains governance mechanisms that ensure a smooth exchange and that the rights of the participants are guaranteed. The governance mechanisms are based on international norms, laws and standards which are agreed between the participants.
- **Data Sovereignty:** An important core mechanism for enabling trust is that participants are in control of their data at all times.

[2]https://projects.eclipse.org/projects/technology.edc

[3]https://github.com/eclipse-edc/MinimumViableDataspace

- **Openness:** At its core, a dataspace enables a large number of participants to share data with each other. It also avoids lock-in effects.
- **Federation/Interoperability:** Dataspaces are decentralised, which is described by the term federated. This means that a dataspace can be linked to other dataspaces, but also embedded in them. Open standards are used to achieve this.

Figure 1 illustrates the different roles in a dataspace and how they interact. As mentioned earlier, one of the principles is data sharing, which takes place between the data provider and the data consumer, as shown in the figure. In addition, the role of the federator is needed to implement trust and governance in the dataspace. The federator is also responsible for cataloging and brokering the data sources. Due to the decentralised nature of dataspaces there is no central data repository where the provider stores the data. The consumer can only gain access through the data provider which ensures that the data provider regains control over its data. This fulfils the principle of data sovereignty [2, p. 8].

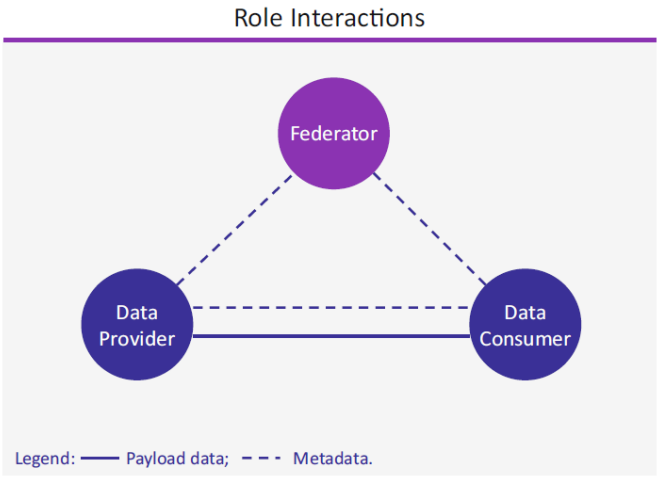


Fig. 1. Role interactions in a dataspace (Source: [2])

A dataspace consists of various components, each with different responsibilities. These components are explained below. A technical explanation of the operational principles based on the MVD is given in section II-B. Figure 2 gives an overview of the responsibilities of the control plane and the data plane. The control plane publishes the catalog and requests the catalogs of the providers. As figure 3 illustrates, a provider can have a dedicated catalog server for publishing a catalog. Also contract agreements and data transfers are negotiated between control planes as figure 2 shows. It contains the policy engine and ensures the policy compliance [7].

The responsibility of the data plane is to handle data transmission. It is possible to deploy the control plane and data plane as one application or as separate applications. To separate concerns it is recommended to deploy the control plane and the data plane as separate applications. Furthermore a control plane can manage multiple data planes. The data plane registers itself with a control plane and informs about the supported wire protocols and data source types. These provided metadata are used by the control plane to determine the available transfer types for an asset and to select a data plane for a transfer process request [8].

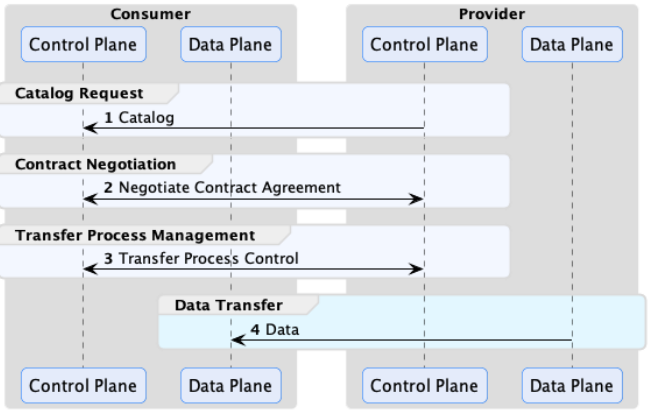


Fig. 2. Control Plane Overview (Source: [7])

The identity hub is like a wallet that stores identity resources of the dataspace participants in an organization. As shown in figure 3 both the consumer and provider have their own identity hub. To be more precise the identity hub stores and manages the Verifiable Credentials (VCs)[4] in a secure way and it is responsible for handling presentation and issuance. It has to be noted that issuance is currently a work in progress in the EDC which leads to the necessity of a dataspace issuer as shown in figure 3 [9].

The catalog server is similar to a control plane but with less dependencies. Its purpose is to provide the catalog that contains assets and linked assets. Linked assets are assets that contain a reference to the control plane that contains the actual asset information. In figure 3 the catalog server contains references to the qna control plane and the manufacturing control plane. Both control planes provide an endpoint to request their catalog containing the asset information. [10]

2) Importance of Dataspaces for Data Ecosystems: Section II-A1 introduces the concept of dataspaces and demonstrates how data can be shared between participants. This forms the basis for demonstrating the importance of dataspaces for data ecosystems. A data ecosystem describes a system that enables the extraction of value from data value chains that interact with organizations and individuals. This system faces challenges such as how to support data sharing. This raises the need to bring together data from multiple participants [11, p. 23].

When it comes to the value of data, as mentioned in section I-A, AI plays an important role. Due to the amount of data being produced and recent improvements in computing power, an increasing amount of big data can be analysed anytime, anywhere. AI increases the value created from big data and

[4]https://www.w3.org/TR/vc-data-model-2.0/

established industrial players have started to implement AI in different areas of industrial applications. Many of these AI applications rely on data-driven approaches such as deep learning [12, p. 381]. The further development of AI, but also of data and robotics technologies, requires the availability of large amounts of data that are cross-sectoral, unbiased, of high quality and trustworthy. One of the keys to unlocking the potential of data that meets these attributes is dataspaces [12, p. 393]. To reflect this need the european data strategy was revised in 2020 to include common european dataspaces, which will ensure that more data becomes available while keeping companies and individuals in control of their data [12, p. 396].

In alignment with the goals of the european data strategy, data sharing has also gained strategic importance for manufacturing ecosystems. [13] conducted a study where they interviewed participants about the strategic relevance of data sharing for manufacturing ecosystems and the perception from a manufacturing ecosystem perspective. In their conclusion they state that based on the interviews the strategic relevance of data sharing is already high and that the general perception is that it is growing in the future. Although the relevance exists they also mention that the principal company is less interested in the concept of opendata whereas technology providers and suppliers see more benefit in sharing data. Technology providers and suppliers see benefits in the form of interoperability towards multiple clients, which [13] explain reflects different perceptions of interest.

3) Use Cases and Real-World Applications: There are already a number of real-world applications that implement the dataspace concept to solve various use cases. One example is the GATE Institute's Urban Data Space[5], which covers the use cases of air quality prediction and social facilities coverage. For air quality prediction the dataspace provides data for a real-time prediction of the air quality in the city Sofia in Bulgaria. For the latter use case data is provided to decision makers to the coverage of social facilities such as schools, hospitals or clinics. The urban dataspace includes data management, analysis and visualisation which makes it covering the entire data value chain. It also compiles with open standards such as interoperability and acts as a gateway to existing EU infrastructures [14, pp. 12–13].

The agricultural sector is also affected by digital transformation [15, p. 279], leading to the Fraunhofer Institute's Cognitive Agriculture lighthouse project[6] [14, p. 13]. The goal of the project is to facilitate the availability of different types of data, such as environmental, operational, machine and process data in digital farms [14, p. 13], as there is potential for improvement in work processes, higher-level planning and decision-making processes [15, p. 280]. There are already existing platforms and systems, but although the integration of platforms is progressing, there is still no connectivity between all these components [15, p. 282]. The dataspace of the Cognitive Agriculture lighthouse project connects these different solutions to enable decision making along the agricultural value chain [14, p. 13]. The following use cases where an agricultural dataspace can add value are identified by [15]: assessing environmental and economic sustainability through the nutrient cycle, exploring new business models and fulfilling regulatory obligations with data, and connecting government platforms to make government data available to the agricultural sector.

Mobility is also a field that also requires large amounts of data to support decision-making. Until now, it has not been possible to process and use the mobility data already collected because of technical, legal or economic reasons. Different existing data platforms are connected through the MDS, making available large amounts of real-time traffic data and sensitive data [16, p. 343]. The MDS is an open data ecosystem that allows providers to define and control the conditions under which their data can be used and exploited by consumers. This provides data providers with a trusted environment that creates data sovereignty and reassures consumers about the quality and provenance of the data. In addition, data sovereignty makes it possible to exploit data generated by vehicles or smartphones and movement patterns that are sensitive in nature [16, p. 345]. This sovereignty is guaranteed by a usage control layer within the connector. It enables the recording of all steps taken during the use and processing of data within the dataspace and gives data providers full knowledge of all activities of their data [16, p. 346]. The open and decentralised architecture used by the MDS was originally developed by the IDSA. This foundation creates new business opportunities for developers and IT service providers. Developers can build mobility services and applications, and IT service providers can use the MDS to host components and data applications in cloud environments [16, p. 346]. Current use cases realised with the MDS include improving the safety of cyclists by using digital technologies and optical signals, integrating different data sources into an ecosystem to create more efficient and sustainable cities, and using car sensor data for traffic planning, police and rescue operations. [17].

Besides existing dataspace implementations for different domains there are already companies that offer different services related to dataspaces. The company [18] offers its own connector, the sovitiy Connector, which is based on the EDC. They also offer Connector-as-a-Service to provide an easy access to dataspaces. There are different packages available that include a managed connector in the cloud but also a community edition[7] that is open source on GitHub[8] and provides the option to host the connector on premise [18]. At the time of writing this thesis version 12.0.0 of the community edition has been released. It is a self-hosting sovity connector and provides a web frontend for managing contracts, data offers, policies and assets. It also includes a transfer history and a catalog browser for finding assets within the dataspace [19].

[5]https://idsa-bulgaria.gate-ai.eu/?page_id=125
[6]https://www.iese.fraunhofer.de/en/project/cognitive-agriculture.html
[7]https://github.com/sovity/edc-ce
[8]https://github.com/

B. Technical Background

To explain how a dataspace works the MVD with EDC version 0.11.0 will be taken as example. By default the dataspace can be deployed either on kubernetes or started locally with IntelliJ. As mentioned in section I-B the kubernetes deployment does not reflect a realistic deployment. Therefore a deployment on Virtual Machines (VMs) was implemented. In the following paragraphs only the most relevant parts are described, for a more detailed explanation [10] provide a readme with sequence diagrams that showcases the interactions between the components in the dataspace.

Figure 3 illustrates the dataspace deployed on VMs and shows the components that are deployed as Docker containers on each VM. The issuer hosts an nginx[9] webserver that serves the Decentralized Identifier (DID) of the issuer. Additionally a Jaeger[10] Docker[11] container runs on this host to collect traces of the datapsace components. The consumer hosts the following dataspace components: a Secure Token Service (STS), identity hub, control plane and data plane. The provider hosts the same components as the consumer but additionally with an catalog server and two control planes and data planes [10].

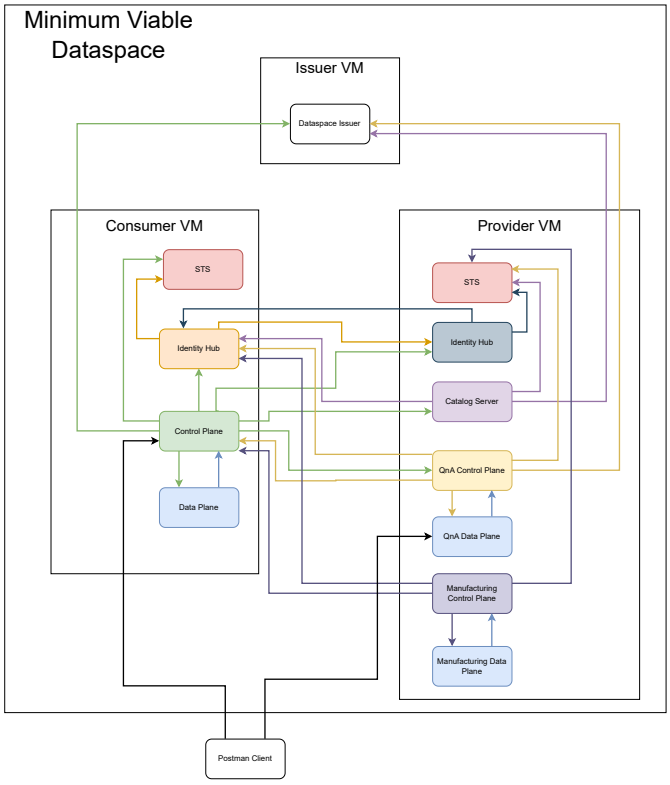


Fig. 3. Overview of the Minimum Viable Dataspace deployed on Virtual Machines (Source: [10])

Based on the system architecture shown in figure 3 the interactions between the dataspace components explained in section II-A1 are explained below.

The interactions will be explained based on the postman requests that are executed. The postman requests are from the point of view of a consumer and are sent to the consumer control plane. In each request there is always a token retrieved and a participant verification done [10]. Therefore these two will be explained at first and omitted in further explanations of component interactions.

To understand the token retrieval we will use the case where the consumer wants to get the catalog from the provider. See [10] for a sequence diagram that visualizes the following explanation. The flow for the catalog request involves several key steps: The consumer control plane retrieves the STS client secret from the consumer vault. Using this secret, it acquires an access token from the consumer STS. The STS then fetches the key to sign the token from the consumer vault. Finally, the consumer control plane sends a catalog request together with the token to the provider's catalog server [10].

Continuing with the catalog request the provider verifies the consumer participant through the provided access token. After retrieving the catalog request the flow on the provider side consists of the following steps: The provider catalog server retrieves its STS client secret from the provider vault. It uses this secret to get a token from the provider STS. The provider catalog server then calls the consumer's identity hub to verify the token presented by the consumer. The consumer identity hub retrieves the key from the consumer vault to verify the token. Upon successful verification, the provider catalog server returns a catalog to the consumer [10].

A request that involves several components of the MVD is the contract negotiation request. According to [10], the procedure unfolds as follows: The Postman client sends a contract negotiation request to the consumer control plane via `/api/management/v3/contractnegotiations`. The consumer control plane internally initiates the contract negotiation process and stores a new contract negotiation record in the consumer Postgres database. It then processes the negotiation in its initial state, updating the contract negotiation record in the consumer database and moving the negotiation into the requesting state. A contract negotiation request is sent to the provider QnA control plane via `/api/dsp/negotiations/request`, which acknowledges the request with a requested notification and inserts a new contract negotiation record into the provider Postgres database. The negotiation is processed internally in the requested state, updated in the provider database, and moved into the agreeing state. The provider sends a contract agreement message to the consumer at `/api/dsp/negotiations/{id}/agreement`, which the consumer control plane processes by notifying it has agreed. A corresponding contract agreement is stored in the consumer Postgres database, and the consumer updates its contract negotiation record. Meanwhile, the provider updates its contract agreement and negotiation records accordingly, transitioning the negotiation to the verified state and updating the relevant entries in its database.

[9]https://nginx.org/
[10]https://www.jaegertracing.io/
[11]https://www.docker.com/

The provider then moves the negotiation to the finalizing state and sends an event notification to the consumer via `/api/dsp/negotiations/{id}/events`. The provider continues to update the contract agreement and negotiation records as part of finalization. On the consumer side, the negotiation is processed in the agreed state, with the consumer updating its contract agreement and negotiation records and transitioning the negotiation to the verifying state. The consumer control plane sends a verification request to the provider, which the provider QnA control plane confirms by sending a verified notification. The provider updates its contract agreement and negotiation records. Finally, the consumer also updates its contract agreement and negotiation records to reflect the completed state.

III. Conceptualization of a Dataspace Blueprint

This section outlines the solution approach to streamlining the creation process for EDC projects using a dedicated project initializer and solving the deployment problems identified in the previous chapters.

The first section introduces the concept of the project initializer, which acts as a tool for generating starting templates for EDC projects, and outlines the features it includes. It is also explained which technologies are used and why they were chosen.

In the second section two deployment strategies for deploying dataspaces are presented. These strategies are categorised as either simple or flexible. The differences in deployment based on project structure and the tools and strategies used in section IV to achieve an automated and standardized deployment are explained.

A. Project Initializer

Currently the only option to start a new project based on the EDC is the provided repository "Template-Basic"[12] on GitHub. The template offers a good starting point but has a few limitations. It contains a control plane and data plane runtime but in case an identity hub wants to be created it is necessary to add a new runtime to the project with the necessary dependencies. Also the name of the project, the group id and the artifact id have to be changed manually. To provide a better experience for creating a starting template there are already tools like the Spring Initializr[13] for creating spring boot projects. As nothing similar exists for EDC projects, a Proof of Concept (PoC) for a similar tool will be implemented as part of this paper.

In the following paragraph the planned features are explained and a mockup of the User Interface (UI) is presented. The features and the UI are inspired by the Spring Initializr as it is a well known software for generating spring boot project templates. Due to time limitations for this paper only the most relevant features will be implemented to be able to create a new project. The EDC project initializer provides the user the possibility to enter project information such as the project name and the group id for the project. Additionally a list of available EDC versions is provided to select the desired version for the project. Based on the selected version all available Maven packages on Maven Central Repository[14] provided by the Eclipse community are retrieved. From the list of available packages, the dependencies that should be included in the project can be selected. After the project information is entered and the dependencies are selected, the project can be generated and retrieved as a ZIP directory. The ZIP directory contains a Java Gradle project that can be opened with an editor or IDE and includes a Dockerfile.

For the sake of simplicity the generated project is not a Gradle Multi-Project Build[15] project such as the project included in the "Template-Basic" on GitHub. The project contains a single Java Gradle project which contains one EDC component.

Figure 4 shows a mockup of the UI. On the left side is a form to enter the project information and a dropdown to select the EDC version. The selected dependencies are shown as a scrollable list on the right side. Each list item contains an icon to remove it from the list of the selected items. In the top right corner is a button to open the list of available Maven packages and on the bottom is a button to generate the project based on the provided information.

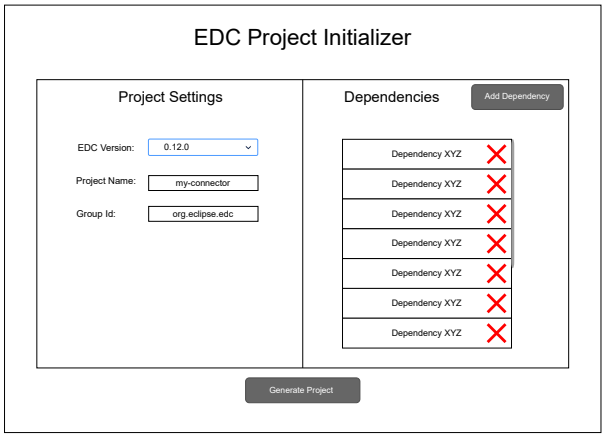


Fig. 4. Mockup of the EDC Project Initializer

Figure 5 shows the modal to select the dependencies. It contains a scrollable list of the available Maven packages for the previously selected EDC version. Below the list the current displayed page is shown and a pagination is provided. Next to the navigation is a dropdown to select how many items should be displayed per page. In the bottom right corner is a button to close the modal.

In the following paragraph the technologies that will be used for the implementation are stated. As the EDC project is an Eclipse project, Quarkus will be used for the backend of the application. Quarkus relies on official standards such

[12]https://github.com/eclipse-edc/Template-Basic
[13]https://start.spring.io/
[14]https://central.sonatype.com/
[15]https://docs.gradle.org/current/userguide/organizing_gradle_projects.html

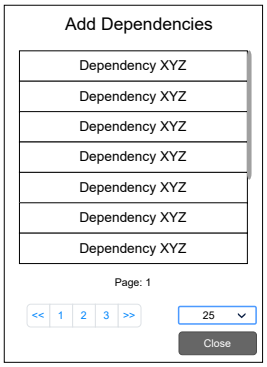


Fig. 5. Mockup of the dependencies modal

as Eclipse MicroProfile [20]. The Eclipse community should be familiar with the eclipse standards. It also contains its own templating engine Qute[16] that will be used for generating the project. React together with the component library MUI[17] has been chosen for building the frontend because the author of this paper has already worked with this technology in the past and React is also one of the most widely used JavaScript Frameworks according to the Stack Overflow Developer Survey 2024 [21]. For development and to start the application locally Docker and Docker Compose will be used. Docker Compose can be used to provide a fixed development environment containing the Java and Node.js versions used for developing the application. It can also be used to start the entire application locally.

B. Deployment Strategy

For the deployment of the dataspace the MVD shown in figure 3 will be reused and deployed on two servers. Currently, the repository contains both the consumer and the provider. To create a scenario in which the consumer and provider are different companies, the repository will be divided into two: one for the consumer and one for the provider. Each repository will have its own deployment.

1) Deployment Considerations for Dataspaces: Different considerations such as the project structure or the flexibility have to be taken into account when deploying a dataspace. The way the project is structured is the first thing to consider. In the MVD all components reside in the same project as a Gradle Multi-Project Build where each component is a subproject. As proposed in section III-A each component can be a single Gradle project. For example the control plane is a Java Gradle project as well as the data plane. Both approaches have different advantages and disadvantages. With the Single-Project Build each project is clearly separated from the others whereas the Multi-Project Build offers the possibility to put related projects together and to share code between projects. For the implementation in section IV the Multi-Project Build will be used to keep all components for the consumer but also for the provider together in a single project and GitHub repository.

Another important decision is the flexibility of the deployment and if the components should be scaled independently. Also the possibility for rolling updates to achieve zero downtime can be important. In general, the data plane should be highly available and scalable, as consumers want to be able to access their purchased data at any time. Reliability is already ensured by the EDC components as they are designed to be reliable [22]. Docker Compose is suitable for local development, testing and simple deployments [23]. When deploying the components with Docker Compose it creates a Docker Network by default where each container defined in the Compose file is part of. This ensures that the containers hosting the components can communicate [24]. This is necessary because the data plane needs to register itself on the control plane as explained in section II-A1. To achieve zero downtime when updating components of the dataspace, Kubernetes provides the feature of rolling updates [25]. Kubernetes also helps with achieving high availability through ReplicaSets which can be used for deploying the data plane. With a ReplicaSet a number of running pods can be specified and Kubernetes ensures that the specified number of pods are running at any time [26]. For the implementation of the deployment strategy simple and flexible deployment methods will be used to deploy the components of the dataspace.

2) Tools and Technologies for the Deployment Process: After the deployment considerations the tools and technologies used for the deployment are explained in the following paragraph. For containerization of the components Docker will be used as each component of the MVD project already has a Dockerfile. For the simple deployment Docker Compose has been chosen as it is suitable for simple deployments as mentioned in the previous section. Figure 6 illustrates the simple deployment where all components are version controlled with GitHub and deployed via a CI/CD Pipeline hosted on Azure DevOps[18]. All components will be deployed together via a single pipeline. GitHub is used as a version control system to make the implementation results available to the public and because the entire EDC project is also hosted on GitHub. The decision to use Azure DevOps was made because the servers provided by the Vorarlberg University of Applied Sciences can only be accessed by the CI/CD Pipeline if Azure DevOps is used. For the Kubernetes cluster microk8s[19] is used because it is an official Kubernetes Cluster of the company Canonical. Canonical is the company behind Ubuntu and Ubuntu was already installed on the servers.

The same tools and technologies are used for the flexible deployment, but Kubernetes is used to deploy the components instead of Docker Compose, as shown in figure 7. Each component also has its own pipeline, enabling independent deployment.

[16]https://quarkus.io/guides/qute
[17]https://mui.com/
[18]https://azure.microsoft.com/en-us/products/devops
[19]https://microk8s.io/

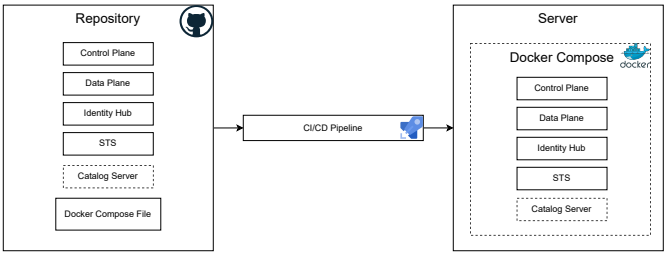


Fig. 6. Simple Deployment

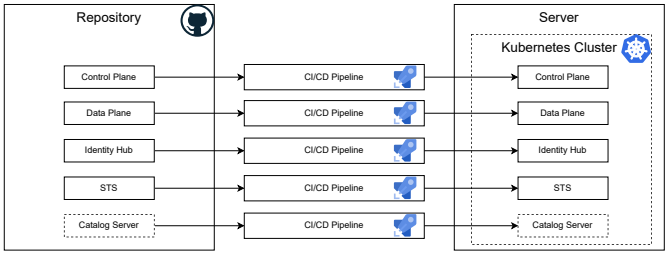


Fig. 7. Flexible Deployment

IV. Implementation of the Blueprint using Eclipse Dataspace Components

The conceptual blueprint was implemented as a reference implementation using EDC. The code is open source and available on GitHub:

- Project Initializer: https://github.com/Smightym8/edc-project-initializer
- Consumer Docker Compose Deployment: https://github.com/Smightym8/mvd-consumer
- Consumer Kubernetes Deployment: https://github.com/Smightym8/mvd-consumer-kubernetes
- Provider Docker Compose Deployment: https://github.com/Smightym8/mvd-provider

Due to page limitations, the technical details of the implementation have been omitted. The repositories include documentation, development set-up instructions and deployment instructions.

V. Evaluation

This section presents the tests performed for the setup and deployment of the dataspace. The results of these tests are then discussed and analysed. Finally, the limitations of the study are outlined.

A. Test Scenarios

This section evaluates the deployment of the dataspace and explains the specific use case applied for the evaluation.

1) Scenario for the Evaluation of the Dataspace Deployment: In order to evaluate the deployment and the created dataspace templates, a dataspace was setup in the Digital Factory Vorarlberg. Also four experts from the Digital Factory used the project initializer and the created templates to setup a dataspace. Section V-B discusses the feedback provided by the experts.

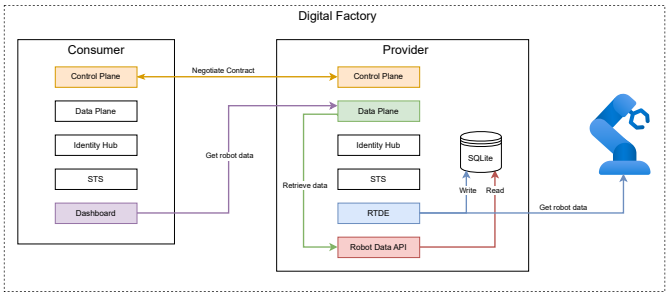


Fig. 8. Overview of the Dataspace for the evaluation

A consumer and a provider were deployed using Docker Compose. Figure 8 shows the structure of the dataspace which is hosted on two computers provided by the Digital Factory Vorarlberg. A Python script named RTDE runs on the provider and collects axle data from a robot. Another Python script provides an Application Programming Interface (API) that is only accessible within the network of the provider and not publicly accessible. For a consumer it is only possible to access the robot axle data through the dataspace.

The use case for the evaluation is as follows: The consumer negotiates a contract with the provider, requesting the asset containing the robot data. After the negotiation of the contract, the consumer receives a token from the provider, which is required to access the robot data via the public API of the provider data plane. The robot data can then be retrieved via the public API of the provider data plane. Although anyone can call the public API of the data plane, it is not possible to get access to any data without a valid token. The consumer hosts a dashboard that polls the provider's data plane for data at specified intervals. Figure 9 shows a screenshot of the dashboard, that displays the retrieved data in a chart. The Digital Factory created a programme that was loaded onto the robot, making it follow a specific sequence. In the screenshots it can be seen that the data is repeated periodically.

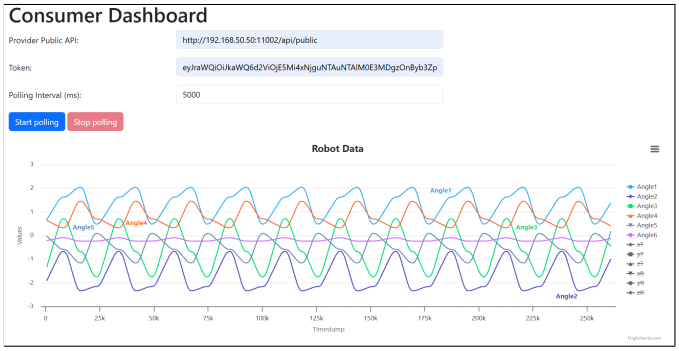


Fig. 9. Consumer Dashboard

B. Results Analysis

The dataspace setup in the Digital Factory demonstrates the entire data value chain, from data collection and provision via a dataspace, to data consumption by services such as the dashboard, which shows how data retrieved via the dataspace can be used. As the dashboard is just an example, other software could use the data provided by the provider. For instance, the data could be used in the future to train an AI model to predict the position of a robot.

As said in section V-A1 four experts used the results of this study to setup a dataspace. Out of the four experts one person has a background in Mechatronics whereas the other three have a background in Computer Science.

The person with the background in Mechatronics studied at the Vorarlberg University of Applied Sciences, already worked with Docker and had some experience in programming. The feedback from this person was positive although it needed some explanation to setup the dataspace. It was stated that especially the project initializer simplified the process of creating a dataspace because prior to the evaluation the person did not know the concept of such a project initializer. This feedback shows that also for people without knowledge in Computer Science are able to setup a dataspace based on the results of this thesis.

The first person with a background in Computer Science also studied at the Vorarlberg University of Applied Sciences. This person also had experience with the EDC through the Digital Factory Vorarlberg. The feedback showed that the instructions provided in the GitHub repository of the templates were clear and no additional explanation was needed for this person.

The second person with a background in Computer Science also studied at the Vorarlberg University of Applied Sciences and worked in the industry before. The feedback was similar as the one of the first person with a background in Computer Science. It was only asked what are further steps after the setup of the dataspace. As the goal of the thesis was to provide a blueprint for building and deploying a dataspace, further steps after the setup are not part of the thesis. Although, in section V-A1 an example is given about how the dataspace can be used for a given use case.

The last person also had a background in Computer Science and studied at the TU Wien. This person had no experience with the EDC and dataspaces. The feedback stated that through following the instruction provided in the GitHub repositories of the templates it was possible to setup the dataspace. This person also mentioned that further steps for using the dataspace would have been interesting.

C. Limitations of the Study

The developed project initializer is only a PoC and is currently limited to creating a Gradle Single-Project Build. A Gradle Multi-Project Build is currently not supported and could be implemented in future. It also does not guarantee that the created project is a functioning dataspace component, as any dependencies can be included in the project, regardless of whether they are compatible or if any additional dependencies are required for it to function.

The dataspace templates created as part of this thesis simplify the setup and deployment of dataspaces, but they are not yet ready for production. The test setup above (see section V-A1) demonstrated that the entire data value chain could be created using the templates, but this remains a demo environment. Additionally, new problems can arise based on the environment in which the dataspace is set up, which makes it error-prone.

VI. Conclusion and Outlook

This section summarises the key findings of this paper and provides an outlook on future research directions in the area of dataspaces.

A. Summary of the Key Findings

This paper details two primary contributions:

- The creation of a PoC project initializer for EDC projects
- Showcasing a simple and flexible deployment approach for dataspaces, as well as providing templates for deployment.

The following paragraphs outline the conclusion for the two contributions. Firstly, the project initializer acts as a PoC, demonstrating how new projects can be created using a dedicated tool. Although it is already possible to create a Single-Project Build, there are still limitations as it is a PoC. Currently, the tool relies on GitHub releases of the EDC connector to provide available versions. The problem with this source is that releases can be overridden or deleted. During the preliminary work for this paper, when the MVD was deployed on two VMs, the dataspace could no longer be built because the version used had been deleted. There is also no documentation or guide explaining which dependencies are needed when creating a project. For example, it is possible to add dependencies for all components to produce one executable containing a control plane, data plane, and identity hub. This issue arises from the flexibility of the EDC, which acts as a toolbox rather than providing an out-of-the-box solution.

The deployment demonstrated that a dataspace can be deployed using simple and flexible methods. It can be concluded that the EDC can be deployed with Docker Compose and Kubernetes without any significant problems or limitations. Furthermore, Kubernetes can be considered a useful deployment tool for achieving zero downtime through rolling updates and ensuring high availability, as recommended by the [22]. Also section V-A1 showed that the entire data value chain can be covered with a dataspace that is setup with the created deployment template. This shows the practical usage of the created deployment templates and the capabilities of the EDC.

The four experts who used the results of this paper to set up a dataspace highlighted that it is easier to set up a basic project (see section V-B). The feedback of the experts shows that those with no prior experience in the field of dataspaces are also able to set up a dataspace with the project initializer and the provided deployment templates.

B. Future Research Directions

There is still a lot to research in the field of dataspaces as the current list of research projects[20] of the IDSA shows. The projects cover topics such as Sustainability, Security and integrating the concepts of dataspaces into different domains.

Based on the contributions of this thesis there is also further research possible. The project initializer could be extended to support the generation of a Gradle Multi-Build Project instead of currently only Gradle Single-Build Project to include all dataspace components in one project. Furthermore, different out-of-the-box templates could be provided through the tool that already contain the dependencies that are needed for example for a control plane.

Another interesting research topic mentioned by [27] are Privacy-Preserving Techniques, which can play a pivotal role a pivotal role in enhancing trust for data-sharing. Earlier this year [28] published a paper analysing mechanisms for privacy preservation and their integration into dataspaces. This demonstrates that this is an actively researched topic with potential for further exploration.

References

[1] M. Franklin, A. Halevy, and D. Maier, "From databases to dataspaces: a new abstraction for information management," *ACM SIGMOD Record*, vol. 34, no. 4, pp. 27–33, Dec. 2005. [Online]. Available: https://dl.acm.org/doi/10.1145/1107499.1107502

[2] B. Otto, "The Evolution of Data Spaces," in *Designing Data Spaces*, B. Otto, M. Ten Hompel, and S. Wrobel, Eds. Cham: Springer International Publishing, 2022, pp. 3–15. [Online]. Available: https://link.springer.com/10.1007/978-3-030-93975-5_1

[3] E. Curry, S. Scerri, and T. Tuikka, "Data Spaces: Design, Deployment, and Future Directions," in *Data Spaces*, E. Curry, S. Scerri, and T. Tuikka, Eds. Cham: Springer International Publishing, 2022, pp. 1–17. [Online]. Available: https://link.springer.com/10.1007/978-3-030-98636-0_1

[4] European Commission, "European Data Governance Act | Shaping Europe's digital future," Oct. 2024. [Online]. Available: https://digital-strategy.ec.europa.eu/en/policies/data-governance-act

[5] ——, "Data Act | Shaping Europe's digital future," Oct. 2024. [Online]. Available: https://digital-strategy.ec.europa.eu/en/policies/data-act

[6] A. Reiberg, C. Niebel, and P. Kraemer, "What is a Data Space?" Gaia-X Hub Germany, Tech. Rep. 1/2022, Sep. 2022. [Online]. Available: https://gaia-x-hub.de/wp-content/uploads/2022/10/White_Paper_Definition_Dataspace_EN.pdf

[7] Eclipse Contributors, "Control Plane," Dec. 2024. [Online]. Available: https://eclipse-edc.github.io/documentation/for-adopters/control-plane/

[8] ——, "Data Plane," Feb. 2025. [Online]. Available: https://eclipse-edc.github.io/documentation/for-adopters/data-plane/

[9] ——, "Identity Hub," Sep. 2024. [Online]. Available: https://eclipse-edc.github.io/documentation/for-adopters/identity-hub/

[10] M. Spiegel and M. Prescher, "Smightym8/MinimumViableDataspace at edc-0.11.0," Apr. 2025. [Online]. Available: https://github.com/Smightym8/MinimumViableDataspace

[11] E. Curry and T. Tuikka, "An Organizational Maturity Model forData Spaces: A Data Sharing Wheel Approach," in *Data Spaces*, E. Curry, S. Scerri, and T. Tuikka, Eds. Cham: Springer International Publishing, 2022, pp. 21–42. [Online]. Available: https://link.springer.com/10.1007/978-3-030-98636-0_2

[12] S. Zillner, J. A. Gomez, A. García Robles, T. Hahn, L. Le Bars, M. Petkovic, and E. Curry, "Data Economy 2.0: From Big Data Value to AI Value and a European Data Space," in *The Elements of Big Data Value*, E. Curry, A. Metzger, S. Zillner, J.-C. Pazzaglia, and A. García Robles, Eds. Cham: Springer International Publishing, 2021, pp. 379–399. [Online]. Available: https://link.springer.com/10.1007/978-3-030-68176-0_16

[13] M. Jurmu, I. Niskanen, A. Kinnula, J. Kääriäinen, M. Ylikerälä, P. Räsänen, and T. Tuikka, "Exploring the Role of Federated Data Spaces in Implementing Twin Transition within Manufacturing Ecosystems," *Sensors*, vol. 23, no. 9, p. 4315, Apr. 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/9/4315

[14] M. Bacco, A. Kocian, S. Chessa, A. Crivello, and P. Barsocchi, "What are data spaces? Systematic survey and future outlook," *Data in Brief*, vol. 57, p. 110969, Dec. 2024. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2352340924009314

[15] R. Kalmar, B. Rauch, J. Dörr, and P. Liggesmeyer, "Agricultural Data Space," in *Designing Data Spaces*, B. Otto, M. Ten Hompel, and S. Wrobel, Eds. Cham: Springer International Publishing, 2022, pp. 279–290. [Online]. Available: https://link.springer.com/10.1007/978-3-030-93975-5_17

[16] S. Pretzsch, H. Drees, and L. Rittershaus, "Mobility Data Space: A Secure Data Space for the Sovereign and Cross-Platform Utilization of Mobility Data," in *Designing Data Spaces*, B. Otto, M. Ten Hompel, and S. Wrobel, Eds. Cham: Springer International Publishing, 2022, pp. 343–361. [Online]. Available: https://link.springer.com/10.1007/978-3-030-93975-5_21

[17] DRM Datenraum Mobilität GmbH, "Mobility Data Space – the Data Sharing Community," Apr. 2025. [Online]. Available: https://mobility-dataspace.eu/use-cases

[18] sovity GmbH, "Connect to Data Space: Secure usage of data with partners," Feb. 2025. [Online]. Available: https://sovity.de/en/connect-to-data-space-en/

[19] ——, "sovity/edc-ce," Apr. 2025. [Online]. Available: https://github.com/sovity/edc-ce

[20] Quarkus Project, "Standards - Quarkus," Mar. 2025. [Online]. Available: https://quarkus.io/standards/

[21] Stack Exchange Inc., "Technology | 2024 Stack Overflow Developer Survey," May 2024. [Online]. Available: https://survey.stackoverflow.co/2024/technology#1-web-frameworks-and-technologies

[22] Eclipse Contributors, "Distributions, Deployment, and Operations," Nov. 2024. [Online]. Available: https://eclipse-edc.github.io/documentation/for-adopters/distributions-deployment-operations/

[23] Docker Inc., "Why use Compose?" Sep. 2024. [Online]. Available: https://docs.docker.com/compose/intro/features-uses/

[24] ——, "Networking," Mar. 2025. [Online]. Available: https://docs.docker.com/compose/how-tos/networking/

[25] The Kubernetes Authors, "Performing a Rolling Update," Nov. 2024, section: docs. [Online]. Available: https://kubernetes.io/docs/tutorials/kubernetes-basics/update/update-intro/

[26] ——, "ReplicaSet," Apr. 2025, section: docs. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/

[27] L. Dutkiewicz, Y. Miadzvetskaya, H. Ofe, A. Barnett, L. Helminger, S. Lindstaedt, and A. Trügler, "Privacy-Preserving Techniques for Trustworthy Data Sharing: Opportunities and Challenges for Future Research," in *Data Spaces*, E. Curry, S. Scerri, and T. Tuikka, Eds. Cham: Springer International Publishing, 2022, pp. 319–335. [Online]. Available: https://link.springer.com/10.1007/978-3-030-98636-0_15

[28] N. B. García, M. H. Padilla, and A. F. S. Gómez, "Privacy Preserving Enablers for Data Space Ecosystems," in *Global Internet of Things and Edge Computing Summit*, M. Presser, A. Skarmeta, S. Krco, and A. González Vidal, Eds. Cham: Springer Nature Switzerland, 2025, vol. 2328, pp. 73–88, series Title: Communications in Computer and Information Science. [Online]. Available: https://link.springer.com/10.1007/978-3-031-78572-6_5

[20] https://internationaldataspaces.org/make/projects/